



MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL

A Constituent Institution of Manipal University

EMBEDDED SYSTEMS

Mini Project

Submitted By:

Aditya Shah-210911310

Jyotishman Shandilya-210911330

Kajal Madhusudan-210911334

BTECH (IT-Batch B2) -V SEMESTER, B
SECTION, DEPARTMENT OF I&CT, MIT,
MANIPAL

TABLE OF CONTENTS

S.No.	TITLE	Page No.
1.	Problem Statement	3
2.	Components Required	3
3.	Circuit Diagram	4
4.	Code	5
5.	Output	9
6.	Conclusion	11
7.	Plagiarism Report	12

PROBLEM STATEMENT

Write a program to interface LPC1768(MBED) with an LM35 temperature sensor for the purpose of temperature monitoring. Implement functionality to acquire temperature readings from the sensor and subsequently modulate the speed of a stepper motor acting as a fan, based on the recorded temperature values.

COMPONENTS REQUIRED

1) LPC1768(MBED KIT)

The LPC1768, an ARM 32-bit CortexM3 Microcontroller developed by NXP (formerly Philips), serves as the core of this system. The code, crafted in Embedded C, underwent rigorous testing on an ALS evaluation board. To translate voltage values into their respective degrees Celsius, a 12-bit internal Analog-to-Digital Converter has been seamlessly integrated into the system.

2) LM35 SENSOR

LM35 sensors are highly accurate integrated-circuit temperature devices that produce an output voltage directly proportional to the temperature in degrees Celsius. The linearity is maintained at a scale factor of 10.0 mV/°C. These sensors offer a wide temperature range spanning from -55 degrees to 150 degrees Celsius.

3) STEPPER MOTOR

The stepper motor employed in this project, is a pivotal component of the system designed for dynamic temperature sensing. Integrated seamlessly with the code written in Embedded C for the NXP LPC1768 Microcontroller, the stepper motor serves as a responsive fan. Tested on an ALS evaluation board, the system utilizes the stepper motor to modulate its speed based on temperature readings.

4) CONNECTORS

For connections we have used 1 FRC cable and 3 Female to Female connectors (jumper wires) to connect LM35 with P1.31, common Vcc and common GND. The FRC cable connects CNB port to the LCD as well as the Stepper motor alternatively to show the working of the same.

CIRCUIT DIAGRAMS

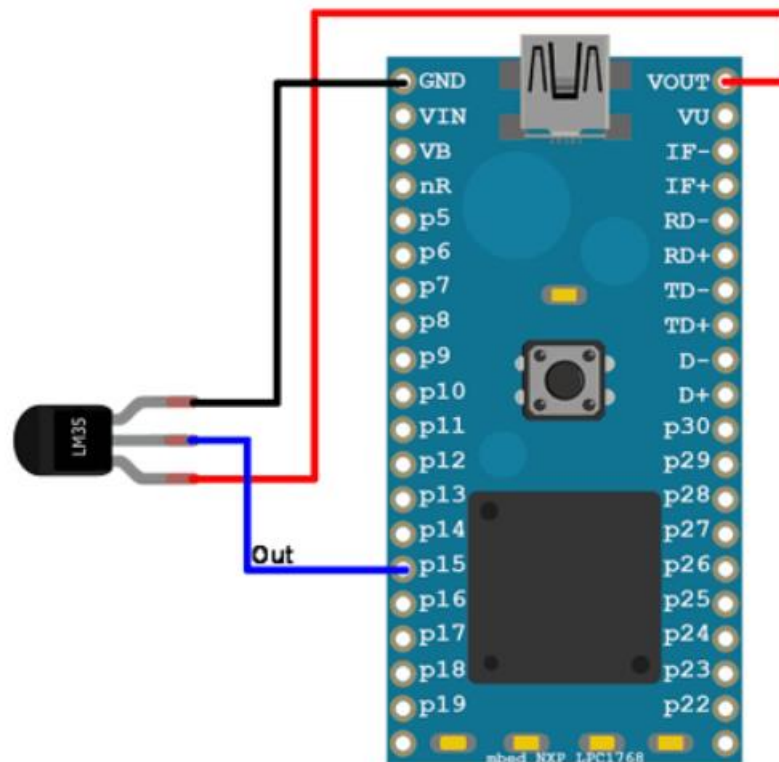


Fig1: Interfacing LM35 With ARM MBED

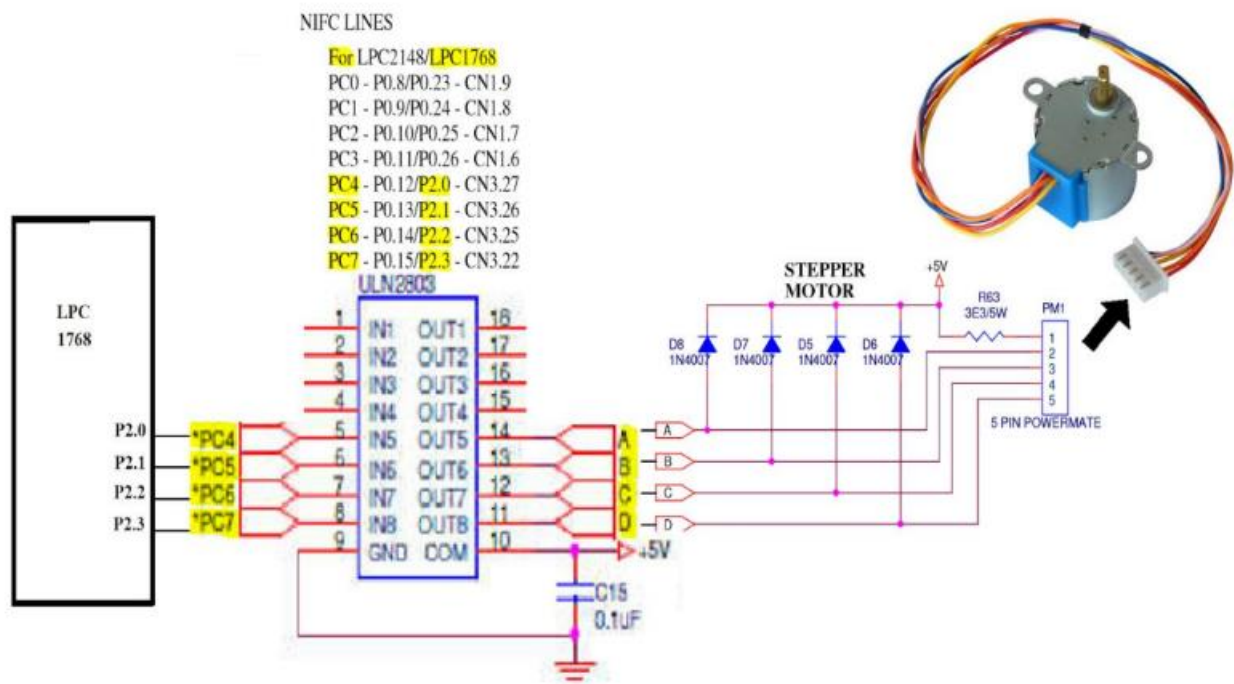


Fig2: Interfacing Stepper Motor with ARM MBED

CODE

```
#include <LPC17xx.h>
#include <stdio.h>
#include <math.h>
#include <stdlib.h>

#define ref_volt 3.3
#define rsctrl 0x00000100 // P0.27
#define enctrl 0x00000200 // P0.28
#define dtctrl 0x000000F0 // P0.23 to P0.26 data lines

void lcd_initialize(void);
void write_cmd(void);
void cleardisplay(void);
void delay(unsigned int);
void lcd_com(void);
void write_data(void);
void lcd_data(void);
void clear_ports(void);
void lcd_puts(unsigned char *);
unsigned int i = 0, j = 0, k = 0;
unsigned long int init_commands[] = {0x30, 0x30, 0x30, 0x20};
unsigned long int additional_commands[] = {0x28, 0x0C, 0x06, 0x01, 0x80};
unsigned long int var1 = 0, var2 = 0;
unsigned long int val = 23;
float t2 = 0, res = 0;
unsigned int i, j, flag;
unsigned char message[10] = {"ADC :"};
unsigned int step_pos[] = {3, 9, 0xC, 6};

void clear_ports(void)
{
    // Clearing the lines at power on
    LPC_GPIO0->FIOCLR = dtctrl; // Clearing data lines
    LPC_GPIO0->FIOCLR = rsctrl; // Clearing RS line
    LPC_GPIO0->FIOCLR = enctrl; // Clearing Enable line
    return;
}

void write_data(void)
{
    clear_ports();
    LPC_GPIO0->FIOPIN = var2; // Assign the value to the data lines
    LPC_GPIO0->FIOSET = rsctrl; // Set bit RS
    LPC_GPIO0->FIOSET = enctrl; // EN=1
    delay(25);
    LPC_GPIO0->FIOCLR = enctrl; // EN =0
    return;
}
```

```

void write_cmd(void) // Write command reg
{
    clear_ports();
    LPC_GPIO0->FIOPIN = var2;    // Assign the value to the data lines
    LPC_GPIO0->FIOCLR = rsctrl;  // Clear bit RS
    LPC_GPIO0->FIOSET = enctrl;  // EN=1
    delay(2500);
    LPC_GPIO0->FIOCLR = enctrl;  // EN =0
    return;
}

void lcd_data(void)
{
    var2 = var1 & 0xf0;
    var2 = var2;
    write_data();
    var2 = var1 & 0x0f;
    var2 = var2 << 4;
    write_data();
    delay(100000);
    return;
}

void lcd_com(void)
{
    var2 = var1 & 0xf0; // Move data (26-8+1) times: 26 - HN
    var2 = var2;        // Place, 4 - Bits
    write_cmd();
    var2 = var1 & 0x0f; // 26-4+1
    var2 = var2 << 4;
    write_cmd();
    delay(100000);
    return;
}

void lcd_initialize()
{
    // Ports initialized as GPIO
    LPC_PINCON->PINSEL3 &= 0xFC003FFF; // P0.23 to P0.28
    // Setting the directions as output
    LPC_GPIO0->FIODIR |= dtctrl;
    LPC_GPIO0->FIODIR |= rsctrl;
    LPC_GPIO0->FIODIR |= enctrl;
    clear_ports();
    delay(320000);
    for ( i = 0; i < 4; i++)
    {
        var2 = (init_commands[i] << 19);
        write_cmd();
        delay(3000000);
    }
    for ( i = 0; i < 5; i++)
    {
        var1 = additional_commands[i];
    }
}

```

```

    lcd_com();
    delay(3000000);
}
return;
}
void anti_clock_wise1 (){
    var1 = 0x00000100;
    for(i=0;i<=3;i++){
        var1 = var1>>1;
        LPC_GPIO0->FIOPIN = var1;
        for(k=0;k<9000;k++); // for step speed variation
    }
    return;
}
void motor1(){
    LPC_PINCON->PINSEL0 = 0xFFFF00FF;
    LPC_GPIO0->FIODIR = 0x000000F0;
    for (j = 0; j < 100; j++){
        anti_clock_wise1();
    }
    return;
}
void anti_clock_wise2 (){
    var1 = 0x00000100;
    for(i=0;i<=3;i++){
        var1 = var1>>1;
        LPC_GPIO0->FIOPIN = var1;
        for(k=0;k<45000;k++);
    }
    return;
}
void motor2(){
    LPC_PINCON->PINSEL0 = 0xFFFF00FF;
    LPC_GPIO0->FIODIR = 0x000000F0;
    for (j = 0; j < 75; j++){
        anti_clock_wise2();
    }
    return;
}
void anti_clock_wise3 (){
    var1 = 0x00000100;
    for(i=0;i<=3;i++){
        var1 = var1>>1;
        LPC_GPIO0->FIOPIN = var1;
        for(k=0;k<15000;k++);
    }
    return;
}
void motor3(){
    LPC_PINCON->PINSEL0 = 0xFFFF00FF;
    LPC_GPIO0->FIODIR = 0x000000F0;

```

```

    for (j = 0; j < 75; j++){
        anti_clock_wise3();
    }
    return;
}

void ADC_IRQHandler(void)
{
    t2 = (LPC_ADC->ADDR0 & (0xFFFF << 4) >> 4);
    t2 = (t2 / 12.41) / 12;
    res = t2;
    if (res < val)
        return;
    sprintf(message, "%f", res);
    var1 = 0xc0;
    lcd_com();
    delay(80000);
    lcd_puts(&message[0]);

    if (res >= 18 && res <=23.9)
        motor2();
    else if(res >= 24 && res <=25.9)
        motor3();
    else if (res >= 26 && res <=35)
        motor1();

    return;
}

void delay(unsigned int r1)
{
    unsigned int r;
    for (r = 0; r < r1; r++);
    return;
}

void lcd_puts(unsigned char *buff)
{
    unsigned int i = 0;
    while (buff[i] != '\0')
    {
        var1 = buff[i];
        lcd_data();
        i++;
        if (i == 16)
        {
            var1 = 0xc0;
            lcd_com();
        }
    }
    return;
}

int main(void){
    lcd_initialize();

```



```

LPC_PINCON->PINSEL1 = 1 << 14; // P1.31 as AD0.5
LPC_SC->PCONP |= (1 << 12);
LPC_ADC->ADCR = (1 << 0 | 1 << 24 | 1 << 21);
LPC_ADC->ADINTEN = (1 << 0);
flag = 1;
var1 = 0x80;
lcd_com();
delay(80000);
NVIC_EnableIRQ(ADC_IRQn);
while (1)
{
    LPC_ADC->ADCR = (1 << 0 | 1 << 24 | 1 << 21);
}
}

```

OUTPUT

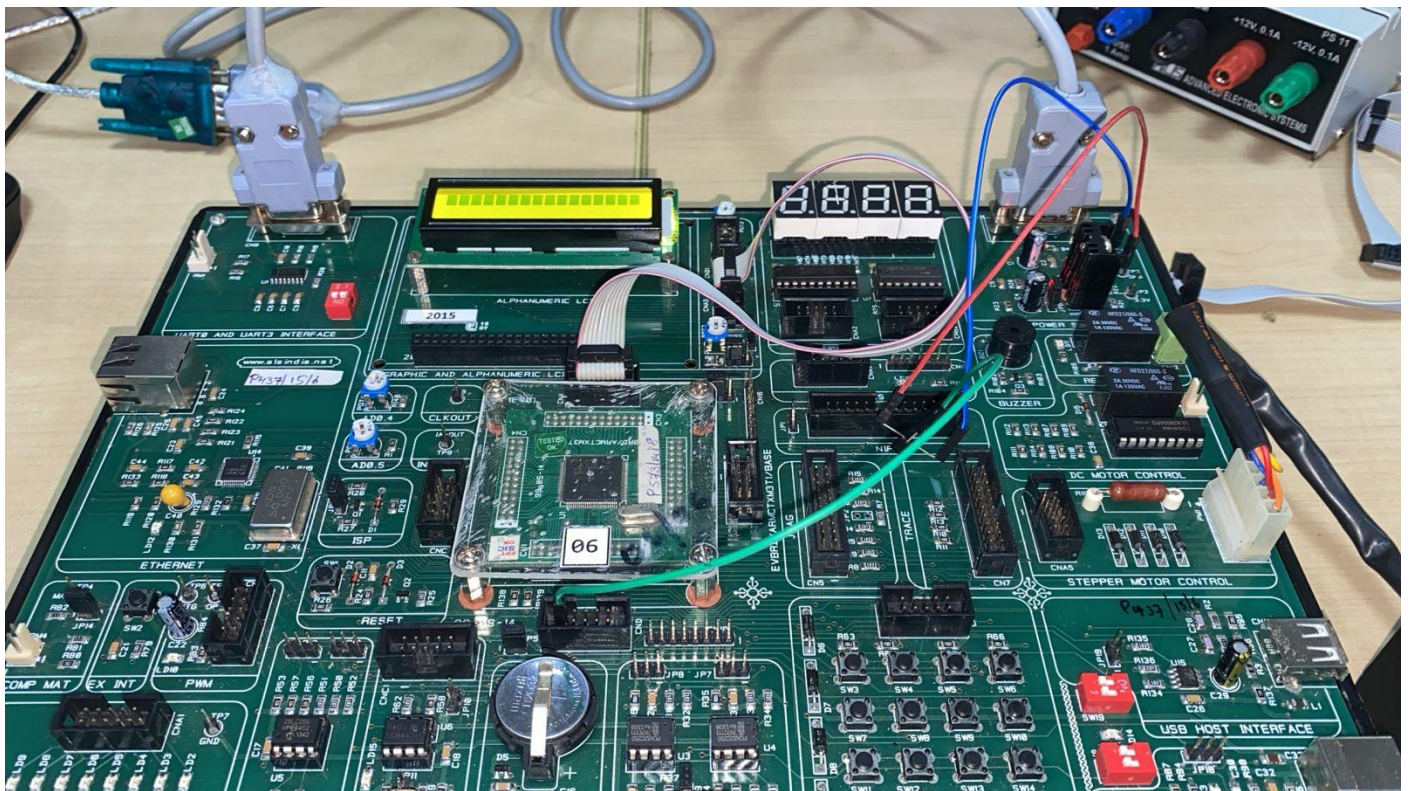


Fig3: Connections made on the LPC1768 MBED KIT and



Fig4: Temperature Output

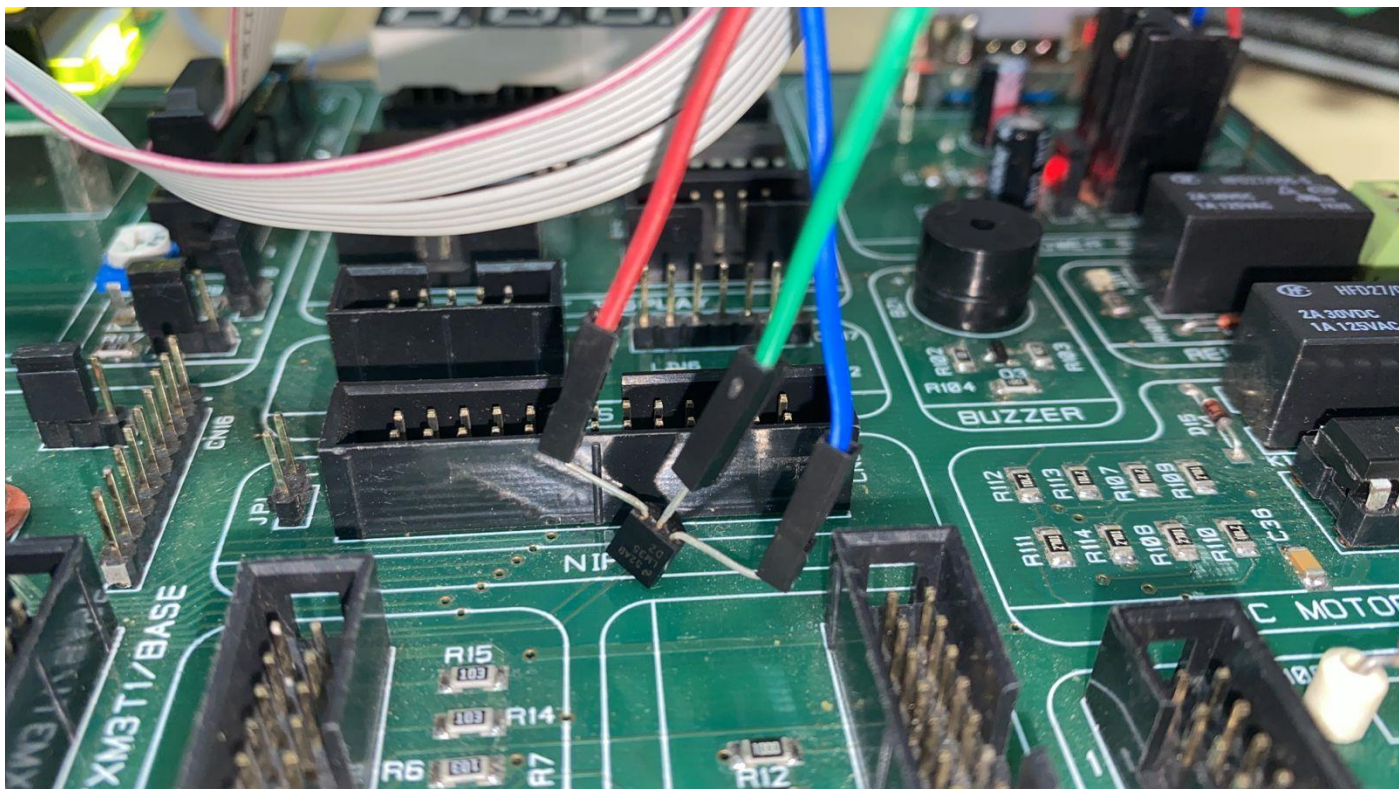


Fig5: LM35 interfacing with LPC1768 MBED KIT using jumper cables



Fig6: Stepper Motor interfaced with LPC1768 MBED KIT

CONCLUSION

Temperature, a measure of the average kinetic energy of particles within a system, undergoes variation in response to changes in molecular movement energy. The addition of heat elevates molecular motion, resulting in a corresponding rise in temperature. Directly quantifying the energy of molecular movement proves challenging, necessitating the use of temperature sensors designed to measure properties responsive to temperature alterations. Our project centers on employing an LM35 sensor to measure temperature and subsequently displaying the results on the terminal. Temperature sensing methods encompass both direct contact with the heat source and non-contact methods utilizing radiated energy. In addition to temperature measurement, we have integrated a stepper motor into the system, functioning as a fan to regulate temperature dynamically. Our circuit consistently yields accurate temperature readings, exhibiting responsiveness to the ambient temperature variations. The recorded temperature fluctuates, maintaining an average of 24.5°C, reflective of the room temperature in the presence of an operating air conditioner. Intriguingly, when we introduced a hairdryer to the sensor, a slight increase in fluctuation of temperature was observed, underscoring the sensor's effectiveness. This dual functionality of temperature sensing and motor-driven fan control enhances the versatility and applicability of our system in temperature regulation scenarios.

Mini-project

ORIGINALITY REPORT

16%

SIMILARITY INDEX

16%

INTERNET SOURCES

2%

PUBLICATIONS

%

STUDENT PAPERS

PRIMARY SOURCES

1

www.coursehero.com

Internet Source

7%

2

shankarrajabagopal.github.io

Internet Source

5%

3

www.msec.ac.in

Internet Source

2%

4

bietdvg.edu

Internet Source

2%

5

pubhtml5.com

Internet Source

1%

Exclude quotes On

Exclude matches < 3 words

Exclude bibliography On